

```
!pip install -qqq langchain-google-genai --progress-bar off
```

```
!pip install langchain_community
```

```

-----
Downloading dataclasses_json-0.6.7-py3-none-any.whl.metadata (25 kB)
Collecting pydantic-settings<3.0.0,>=2.4.0 (from langchain_community)
  Downloading pydantic_settings-2.10.1-py3-none-any.whl.metadata (3.4 kB)
Requirement already satisfied: langsmith>=0.1.125 in /usr/local/lib/python3.11/dist-packages (from la
Collecting httpx_sse<1.0.0,>=0.4.0 (from langchain_community)
  Downloading httpx_sse-0.4.1-py3-none-any.whl.metadata (9.4 kB)
Requirement already satisfied: numpy>=1.26.2 in /usr/local/lib/python3.11/dist-packages (from langcha
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.11/dist-packages (fr
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.11/dist-packages (from aioh
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.11/dist-packages (from aio
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.11/dist-packages (from a
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from aioh
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from aio
Collecting marshmallow<4.0.0,>=3.18.0 (from dataclasses_json<0.7,>=0.5.7->langchain_community)
  Downloading marshmallow-3.26.1-py3-none-any.whl.metadata (7.3 kB)
Collecting typing-inspect<1,>=0.4.0 (from dataclasses_json<0.7,>=0.5.7->langchain_community)
  Downloading typing_inspect-0.9.0-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: langchain-text-splitters<1.0.0,>=0.3.8 in /usr/local/lib/python3.11/di
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.11/dist-packages (fro
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.11/dist-packages (from
Requirement already satisfied: typing-extensions>=4.7 in /usr/local/lib/python3.11/dist-packages (fro
Requirement already satisfied: packaging>=23.2 in /usr/local/lib/python3.11/dist-packages (from langc
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from lang
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in /usr/local/lib/python3.11/dist-packages (from
Requirement already satisfied: requests-toolbelt<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-pack
Requirement already satisfied: zstandard<0.24.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (
Collecting python-dotenv>=0.21.0 (from pydantic-settings<3.0.0,>=2.4.0->langchain_community)
  Downloading python_dotenv-1.1.1-py3-none-any.whl.metadata (24 kB)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-packages (f
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (f
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from re
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from re
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.11/dist-packages (from SQLAlchemy
Requirement already satisfied: anyio in /usr/local/lib/python3.11/dist-packages (from httpx<1,>=0.23.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx<1
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.11/dist-packages (from httpcore==1
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.11/dist-packages (from json
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/dist-packages (fro
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.11/dist-packages (from
Collecting mpy_extensions>=0.3.0 (from typing-inspect<1,>=0.4.0->dataclasses_json<0.7,>=0.5.7->langc
  Downloading mpy_extensions-1.1.0-py3-none-any.whl.metadata (1.1 kB)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio->h
Downloading langchain_community-0.3.27-py3-none-any.whl (2.5 MB)
-----
2.5/2.5 MB 52.3 MB/s eta 0:00:00
Downloading dataclasses_json-0.6.7-py3-none-any.whl (28 kB)
Downloading httpx_sse-0.4.1-py3-none-any.whl (8.1 kB)
Downloading pydantic_settings-2.10.1-py3-none-any.whl (45 kB)
-----
45.2/45.2 kB 5.0 MB/s eta 0:00:00
Downloading marshmallow-3.26.1-py3-none-any.whl (50 kB)
-----
50.9/50.9 kB 5.8 MB/s eta 0:00:00
Downloading python_dotenv-1.1.1-py3-none-any.whl (20 kB)
Downloading typing_inspect-0.9.0-py3-none-any.whl (8.8 kB)
Downloading mpy_extensions-1.1.0-py3-none-any.whl (5.0 kB)
Installing collected packages: python-dotenv, mpy-extensions, marshmallow, httpx-sse, typing-inspect
Successfully installed dataclasses-json-0.6.7 httpx-sse-0.4.1 langchain_community-0.3.27 marshmallow-

```

```

import os
import pandas as pd
from sqlalchemy import create_engine
from langchain_community.utilities import SQLDatabase
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_community.agent_toolkits import create_sql_agent
#from dotenv import load_dotenv
#load_dotenv()

import pandas as pd
from sqlalchemy import create_engine, text

# Load Excel
excel_path = "/content/XONE Deals.xlsx"
data_df = pd.read_excel(excel_path, sheet_name=0)
schema_df = pd.read_excel(excel_path, sheet_name=1)

# Create SQLite database
engine = create_engine("sqlite:///deals.db")

# Save data to SQL
data_df.to_sql("deals", con=engine, if_exists="replace", index=False)

# Print schema
print("Table Schema:")
print(schema_df)

```

➡ Table Schema:

	columnname	type	description
0	codapp	varchar(3)	instrument initials
1	numtra	number(7,0)	deal id
2	uni	varchar(25)	unique trade identifier
3	law	varchar(10)	regulatory law
4	numver	number(5,0)	trade version
5	redid	varchar(25)	red identifier
6	assetclass	varchar(25)	asset class

```

import google.generativeai as genai
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.agents import create_sql_agent
from langchain.agents.agent_toolkits import SQLDatabaseToolkit
from langchain.sql_database import SQLDatabase

# Google API Key
gemini_api_key = "AIzaSyBawA6g_oe-fN_6bgGL_AAN8DAYjsXjI5M"
#genai.configure(api_key=gemini_api_key)

# Connect to database
db = SQLDatabase.from_uri("sqlite:///deals.db")

#Create Genai-llm
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash-exp", temperature=0, google_api_key=gemini_api_key)

# Create SQL agent
toolkit = SQLDatabaseToolkit(db=db, llm=llm)

```

```
agent_executor = create_sql_agent(llm=llm, toolkit=toolkit, verbose=True)
```

```
agent_executor
```

```

↳ AgentExecutor(name='SQL Agent Executor', verbose=True,
  agent=RunnableAgent(runnable=RunnableAssign(mapper={
    agent_scratchpad: RunnableLambda(lambda x: format_log_to_str(x['intermediate_steps']))
  })
  | PromptTemplate(input_variables=['agent_scratchpad', 'input'], input_types={}, partial_variables=
    {'tools': "sql_db_query - Input to this tool is a detailed and correct SQL query, output is a result
    from the database. If the query is not correct, an error message will be returned. If an error is
    returned, rewrite the query, check the query, and try again. If you encounter an issue with Unknown
    column 'xxxx' in 'field list', use sql_db_schema to query the correct table fields.\nsql_db_schema -
    Input to this tool is a comma-separated list of tables, output is the schema and sample rows for
    those tables. Be sure that the tables actually exist by calling sql_db_list_tables first! Example
    Input: table1, table2, table3\nsql_db_list_tables - Input is an empty string, output is a comma-
    separated list of tables in the database.\nsql_db_query_checker - Use this tool to double check if
    your query is correct before executing it. Always use this tool before executing a query with
    sql_db_query!", 'tool_names': 'sql_db_query, sql_db_schema, sql_db_list_tables,
    sql_db_query_checker'}, template='You are an agent designed to interact with a SQL database.\nGiven
    an input question, create a syntactically correct sqlite query to run, then look at the results of
    the query and return the answer.\nUnless the user specifies a specific number of examples they wish
    to obtain, always limit your query to at most 10 results.\nYou can order the results by a relevant
    column to return the most interesting examples in the database.\nNever query for all the columns from
    a specific table, only ask for the relevant columns given the question.\nYou have access to tools for
    interacting with the database.\nOnly use the below tools. Only use the information returned by the
    below tools to construct your final answer.\nYou MUST double check your query before executing it. If
    you get an error while executing a query, rewrite the query and try again.\n\nDO NOT make any DML
    statements (INSERT, UPDATE, DELETE, DROP etc.) to the database.\n\nIf the question does not seem
    related to the database, just return "I don\'t know" as the answer.\n\n{tools}\n\nUse the following
    format:\n\nQuestion: the input question you must answer\nThought: you should always think about what
    to do\nAction: the action to take, should be one of [{tool_names}]\nAction Input: the input to the
    action\nObservation: the result of the action\n... (this Thought/Action/Action Input/Observation can
    repeat N times)\nThought: I now know the final answer\nFinal Answer: the final answer to the original
    input question\n\nBegin!\n\nQuestion: {input}\nThought: I should look at the tables in the database
    to see what I can query. Then I should query the schema of the most relevant
    tables.\n{agent_scratchpad}')
  | RunnableBinding(bound=ChatGoogleGenerativeAI(model='models/gemini-2.0-flash-exp',
    google_api_key=SecretStr('*****'), temperature=0.0, client=
    <google.ai.generative_v1beta.services.generative_service.client.GenerativeServiceClient
    object at 0x783705123550>, default_metadata=(), model_kwargs={}), kwargs={'stop': ['\nObservation']}],
    config={}, config_factories=[])
  | ReActSingleInputOutputParser(), input_keys_arg=['input'], return_keys_arg=['output'],
    stream_runnable=True), tools=[QuerySQLDatabaseTool(description="Input to this tool is a detailed and
    correct SQL query, output is a result from the database. If the query is not correct, an error
    message will be returned. If an error is returned, rewrite the query, check the query, and try again.
    If you encounter an issue with Unknown column 'xxxx' in 'field list', use sql_db_schema to query the
    correct table fields.", db=<langchain_community.utilities.sql_database.SQLDatabase object at
    0x7836c02f2910>), InfoSQLDatabaseTool(description='Input to this tool is a comma-separated list of
    tables, output is the schema and sample rows for those tables. Be sure that the tables actually exist
    by calling sql_db_list_tables first! Example Input: table1, table2, table3', db=
    <langchain_community.utilities.sql_database.SQLDatabase object at 0x7836c02f2910>),
    ListSQLDatabaseTool(db=<langchain_community.utilities.sql_database.SQLDatabase object at
    0x7836c02f2910>), QuerySQLCheckerTool(description='Use this tool to double check if your query is
    correct before executing it. Always use this tool before executing a query with sql_db_query!', db=
    <langchain_community.utilities.sql_database.SQLDatabase object at 0x7836c02f2910>,
    llm=ChatGoogleGenerativeAI(model='models/gemini-2.0-flash-exp',
    google_api_key=SecretStr('*****'), temperature=0.0, client=
    <google.ai.generative_v1beta.services.generative_service.client.GenerativeServiceClient
    object at 0x783705123550>, default_metadata=(), model_kwargs={}), llm_chain=LLMChain(verbose=False,
    prompt=PromptTemplate(input_variables=['dialect', 'query'], input_types={}, partial_variables={},
    template='\n{query}\nDouble check the {dialect} query above for common mistakes, including:\n- Using
  
```

```
#user_input_fetch = "Fetch codapp and numtra of deals having law as HKMA."
user_input_fetch = "Fetch deals deals having law as HKMA."
agent_executor.run(user_input_fetch)
```



> Entering new SQL Agent Executor chain...

Action: `sql_db_list_tables`

Action Input: *deals* I should query the schema of the `deals` table to see what columns I can query.

Action: `sql_db_schema`

Action Input: `deals`

```
CREATE TABLE deals (
  codapp TEXT,
  numtra BIGINT,
  uni TEXT,
  law TEXT,
  numver BIGINT,
  redid TEXT,
  assetclass TEXT
)
```

/*

3 rows from deals table:

	codapp	numtra	uni	law	numver	redid	assetclass
IRD	1234	IRDONE00004321	HKMA	1	xxxx123-yyy456	interest-rate	
IRD	5678	IRDONE000058765	CANADA	2	xxxx375-yyy768	interest-rate	
IRD	7642	IRDONE000062543	EMIR	3	xxxx764-yyy1765	interest-rate	

*/I should query the `deals` table for deals where the `law` is `HKMA`.

Action: `sql_db_query_checker`

Action Input: `SELECT codapp, numtra, uni, law, numver, redid, assetclass FROM deals WHERE law = 'HKMA'`

`SELECT codapp, numtra, uni, law, numver, redid, assetclass FROM deals WHERE law = 'HKMA' LIMIT 10`
 ``````tool\_code```` looks correct.

Action: `sql_db_query`

Action Input: `SELECT codapp, numtra, uni, law, numver, redid, assetclass FROM deals WHERE law = 'HKMA'`

> Finished chain.

'The deals with law HKMA are: IRD, 1234, IRDONE00004321, HKMA, 1, xxxx123-yyy456, interest-rate; IRD, 87623, IRDONE00004127364, HKMA, 2, xxxx52134-yyy76253, interest-rate; IRS, 71265, IRDONE0000982173, HKMA, 1, xxxx6123-yyy76235, interest-rate.'

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

